

Memory-Aware Multi-Sensor Perception for Efficient and Safe Navigation in Dynamic Environments

Anonymous Authors

Abstract—Autonomous navigation in previously unseen environments requires effective perception, persistent environmental representation, and collision avoidance while maintaining progress toward a goal. Existing perception-based methods often rely on prior maps or short-horizon observations, limiting their ability to exploit previously observed structure. We propose a memory-aware multi-sensor navigation framework that integrates LiDAR and RGB perception, online distance-field representation learning, and a stage-adaptive Modulated Control Barrier Function Quadratic Program (MCBF-QP). The framework persistently represents static infrastructure while tracking dynamic obstacles, enabling the MCBF-QP controller to exploit previously observed geometry for obstacle circumvention and adapt its safety constraints and guidance to local conditions. Experiments in complex indoor and outdoor environments demonstrate improved navigation efficiency and goal-reaching performance while maintaining collision avoidance in narrow passages and around dynamic obstacles.

I. INTRODUCTION

Recent advances in autonomous systems have driven substantial progress in robot navigation and collision avoidance. A broad spectrum of approaches has demonstrated strong empirical performance, including graph-search methods, data-driven approaches, sampling-based planners, closed-form reactive strategies, and control optimization [1]–[13]. However, deploying these approaches in previously unseen environments remains challenging because many navigation and safe-control methods rely on prior knowledge of the environment, such as prebuilt maps or externally provided observations. To relax this requirement, perception-based navigation pipelines have increasingly leveraged onboard sensors, including LiDARs and RGB-D cameras, to construct or infer environmental representations online [14]–[20].

Existing perception-based approaches can be broadly distinguished by whether they retain an explicit representation of previously observed environments. Some methods perform long-term navigation using SLAM together with pre-scanned or GPS-referenced maps [14], [15]. While such approaches can exploit global environmental structure, they depend on the availability of a suitable prior map and can become unreliable when the environment changes substantially. Other approaches construct only temporary local representations from onboard sensors to enable reactive path planning in dynamic environments, using either LiDAR [16]–[18] or RGB-D cameras [19]–[21]. These approaches avoid dependence on prior maps and enable fast local reactions, but typically discard useful geometric information once it leaves the local sensing horizon. As a result, the robot may repeatedly react to locally perceived obstacles without exploiting the structure of previously explored free space, increasing the likelihood

of undesirable equilibria, or repeated exploration of already visited regions. We refer to the ability to maintain progress toward the goal while avoiding such behaviors as navigation **liveness**. Persistent representations of previously observed static infrastructure can therefore provide valuable information for improving navigation in the current environment and for subsequent tasks performed in the same environment.

Constructing such persistent representations using only onboard sensing requires robust geometric and semantic perception. As navigation tasks increasingly involve complex and dynamic environments, recent research has increasingly explored LiDAR-camera fusion to combine the complementary strengths of individual sensing modalities [22], [23]. LiDARs provide accurate, long-range geometric measurements with wide angular coverage, while RGB cameras provide rich visual and semantic information for identifying objects and environmental structures [21]. Nevertheless, existing LiDAR-camera fusion pipelines have primarily demonstrated their capabilities in structured environments and have yet to address navigation in complex, unstructured indoor environments that require traversing multiple rooms, reasoning about persistent spatial structure, and avoiding dynamic agents such as pedestrians.

Persistent environmental representation alone, however, does not guarantee successful navigation. The navigation method must also exploit the available environmental information while maintaining both collision avoidance and progress toward the goal. Existing perception-based navigation pipelines have employed a variety of approaches, including deep learning, Timed Elastic Band (TEB), Model Predictive Control (MPC), and Control Barrier Function (CBF)-based controllers [15]–[18]. While these methods have demonstrated effective navigation in various environments, they generally do not provide explicit guarantees of convergence without undesirable equilibria or dysfunctional circulation. Recent work on Modulated Control Barrier Function Quadratic Programs (MCBF-QPs) addresses this limitation by combining collision avoidance with navigation *liveness*, i.e., continued progress toward the goal without undesirable equilibria [12], [13], [24].

These observations motivate a **memory-aware** navigation framework that leverages previously observed environmental structure, rather than relying solely on instantaneous perception, while providing liveness-aware collision avoidance, without requiring an offline map. Thus, the main contributions of this paper are as follows:

- 1) We propose a **multi-sensor** perception and navigation framework for autonomous navigation in complex,

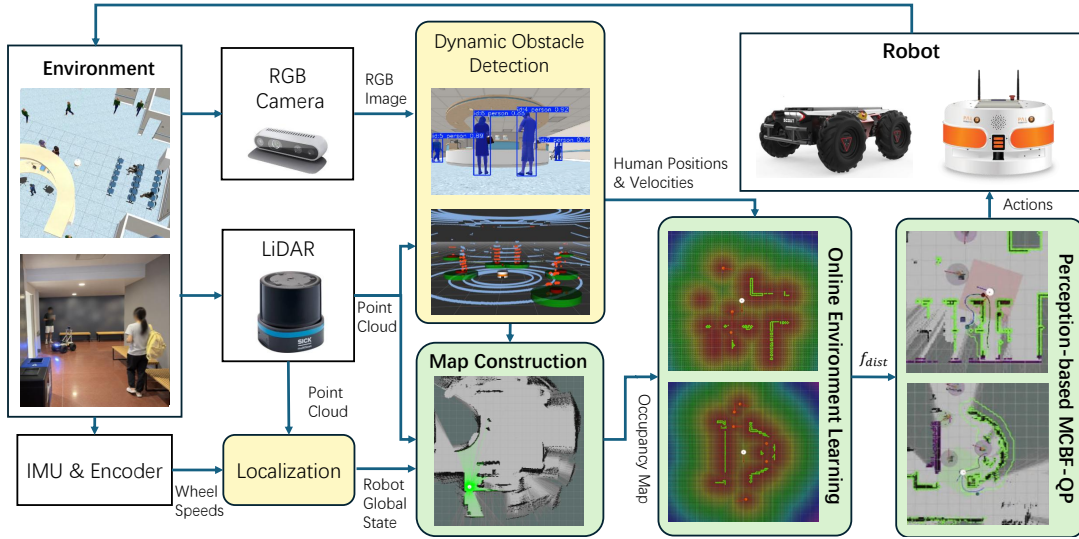


Fig. 1: Overview of the proposed **memory-aware multi-sensory perception and navigation framework**, integrating onboard state estimation, dynamic obstacle detection, online environment representation learning, and stage-adaptive MCBF control for safe and live navigation in previously unmapped environments.

unknown, and unstructured indoor environments.

- 2) We develop an online distance-field-based environmental representation that **persistently** captures static infrastructure while distinguishing dynamic obstacles, without requiring an offline map.
- 3) We integrate the persistent representation with an MCBF-QP-based controller to exploit previously observed environmental structure for obstacle circumvention, and demonstrate **improved mapless navigation** efficiency and goal-reaching performance in dynamic environments.

II. PRELIMINARIES

A. Assumptions

We consider a robotic system governed by the control-affine dynamics

$$\dot{\mathbf{x}} = f(\mathbf{x}) + g(\mathbf{x})\mathbf{u}, \quad (1)$$

where $\mathbf{x} \in \mathbb{R}^n$ is the robot state, $\mathbf{u} \in \mathbb{R}^m$ is the control action, and f and g are Lipschitz continuous functions.

B. Gaussian Process Distance Field

In the proposed pipeline, we use a Gaussian Process Distance Field (GPDF) [25], [26] to obtain a continuous and differentiable distance representation from observed obstacle boundaries. Given a set of obstacle boundary points $\mathcal{P} = \{\mathbf{p}_i\}$, let $o(\mathbf{p})$ denote a latent field modeled as a Gaussian process (GP),

$$o(\mathbf{p}) \sim \mathcal{GP}(0, k_o(\mathbf{p}, \mathbf{p}')), \quad (2)$$

where k_o is selected such that its inverse transformation f_{inv} recovers the Euclidean distance:

$$f_{\text{inv}}(k_o(\mathbf{p}, \mathbf{p}')) := \|\mathbf{p} - \mathbf{p}'\|_2. \quad (3)$$

Assigning a unit latent observation to each boundary point, GP regression gives the posterior mean

$$\bar{o}(\mathbf{p}) = k_o(\mathbf{p}, \mathcal{P}) \left(K_o^{\mathcal{P}} + \sigma_o^2 \mathbf{I} \right)^{-1} \mathbf{1}, \quad (4)$$

where $K_o^{\mathcal{P}} := K_o(\mathcal{P}, \mathcal{P})$ is the covariance matrix evaluated at the boundary points, σ_o is the observation-noise parameter, and $\mathbf{1}$ is a vector of unit observations. The resulting distance estimate is obtained by applying the inverse transformation to the posterior mean:

$$f_{\text{dist}}(\mathbf{p}) = f_{\text{inv}}(\bar{o}(\mathbf{p})). \quad (5)$$

Since both k_o and f_{inv} are differentiable, f_{dist} is differentiable with an analytically computable gradient, making it suitable for constructing the differentiable barrier functions used by the controller.

C. Modulated Control Barrier Functions

To retain CBF-QP safety while enhancing **liveness**, we adopt the on-manifold Modulated CBF-QP (MCBF-QP) as the controller, which provides proven safety and convergence guarantees. MCBF-QP augments the CBF constraint with a tangent-guidance constraint that enforces positive velocity along an estimated obstacle boundary geodesic guiding vector $\phi(\mathbf{x})$, thereby steering the robot along an obstacle-circumventing direction and mitigating undesirable equilibria and dysfunctional circulation [13], [24].

For multiple nearby obstacles, we modify the original MCBF-QP by replacing its unified barrier function with individual barrier functions $h_o(\mathbf{x}, \mathbf{x}_o)$ for each obstacle $o \in \mathcal{O}$. This modification imposes independent safety constraints for each obstacle cluster, yielding a more conservative representation that better preserves clearance from nearby obstacles. The resulting formulation is

$$\begin{aligned} \mathbf{u}_{\text{mcbf}} &= \arg \min_{\mathbf{u} \in \mathbb{R}^m} (\mathbf{u} - \mathbf{u}_{\text{nom}})^\top (\mathbf{u} - \mathbf{u}_{\text{nom}}), \\ \text{s.t.} \quad & \mathcal{L}_f h_o + \mathcal{L}_g h_o \mathbf{u} + \nabla_{\mathbf{x}_o} h_o^\top \dot{\mathbf{x}}_o \geq -\alpha(h_o) \quad \forall o \in \mathcal{O}, \end{aligned} \quad (6)$$

$$\phi(\mathbf{x})^\top f(\mathbf{x}) + \phi(\mathbf{x})^\top g(\mathbf{x})\mathbf{u} \geq v_\phi. \quad (7)$$

Here, $\mathbf{x}_o$ and $\dot{\mathbf{x}}_o$ denote the state and velocity of obstacle o , respectively, $\alpha(\cdot)$ is a class- $\mathcal{K}$ function, and $v_\phi > 0$ is a user-defined constant specifying the minimum velocity component along ϕ . $\mathcal{L}_f, \mathcal{L}_g$ denote the Lie derivatives wrt. each component of the control-affine dynamics in Eq. 1.

III. METHODOLOGY

As shown in Fig. 1, the proposed navigation framework integrates onboard state estimation, multi-sensory perception, online environment representation learning, and stage-adaptive MCBF control to enable collision avoidance and enhance navigation liveness in previously unmapped environments. Onboard state estimation establishes a persistent reference frame, while multi-sensory perception identifies dynamic obstacles and environment geometry from LiDAR and RGB observations (Sections III-A and III-B). The resulting environmental observations are incorporated into an online-learned distance-field representation (Section III-D), which enables the MCBF controller to enforce collision-avoidance constraints and generate obstacle-circumventing guidance. The controller further adapts its CBF constraints and geodesic guidance to the local navigation condition, tailoring its behavior to dynamic obstacles, narrow passages, and open space (Section III-E).

A. Onboard State Estimation

To maintain a persistent environmental representation without an external motion-capture system, sensor observations and the robot state $\mathbf{x}$ are expressed in a common reference frame. The origin of the constructed frame is initialized at the robot's initial position, with the z -axis aligned with the gravity direction and the x - y axes spanning the horizontal plane. The localization pipeline combines the 6-DOF LiDAR-inertial pose from FAST-LIO2 [27] with longitudinal wheel velocity through an extended Kalman filter [28]. LiDAR-inertial odometry provides spatial pose estimation, while wheel odometry provides complementary short-term motion information when LiDAR scan registration is degraded. The fused pose and velocity define the robot state $\mathbf{x}$ in the reference frame and are used to transform LiDAR and camera observations into the same frame for persistent mapping. All mathematical quantities in the following sections are expressed in this reference frame unless otherwise specified.

B. Dynamic Obstacle Detection

Independently movable obstacles, whether in motion or not, pose a higher safety risk than static obstacles and require accurate position and velocity estimates to maintain sufficient clearance and avoid collisions. LiDAR-based dynamic obstacle detection is typically geometry-based [17], [29], but geometric cues alone cannot reliably distinguish movable obstacles from similarly shaped static infrastructure or separate a person from nearby obstacles. In contrast, the YOLO instance-segmentation model [30] provides semantic identification but not reliable metric position or velocity

estimates. LiDAR and RGB observations are therefore jointly adopted in the proposed pipeline, with the two detection branches running in parallel to support accurate identification and tracking of such obstacles.

The LiDAR branch filters $\mathcal{P}_{\text{lidar}}$ along the z -axis to remove floor points and structures well above the robot. The remaining points are clustered and approximated as lines or circles, with line-like clusters treated as likely static infrastructure and circle-like clusters as candidate dynamic obstacles. Their centers are denoted by $\mathbf{p}_{c,k}^{\text{geo}} \in \mathbb{R}^2$.

The camera branch processes RGB images using the YOLO instance-segmentation model [30], producing an instance mask $M_{\text{dyn}}^{(k)}(i, j)$ for each detected instance k , where (i, j) denotes the pixel index. LiDAR points are projected into each image using the corresponding LiDAR-to-camera extrinsic transformation and camera intrinsic matrix. For each instance k , let $\mathcal{J}_k$ denote the set of pixel indices belonging to its instance mask,

$$\mathcal{J}_k = \{(i, j) : M_{\text{dyn}}^{(k)}(i, j) = 1\}. \quad (8)$$

The corresponding mask centroid is

$$(i_c, j_c) = \frac{1}{|\mathcal{J}_k|} \sum_{(i, j) \in \mathcal{J}_k} (i, j). \quad (9)$$

Let $\mathcal{P}_M^{(k)}$ denote the LiDAR points whose projections lie within $M_{\text{dyn}}^{(k)}$. The point whose projected pixel is closest to (i_c, j_c) is derived as

$$\mathbf{p}_{c,k}^{\text{image}} = \arg \min_{\mathbf{p} \in \mathcal{P}_M^{(k)}} \left[(i(\mathbf{p}) - i_c)^2 + (j(\mathbf{p}) - j_c)^2 \right], \quad (10)$$

where $(i(\mathbf{p}), j(\mathbf{p}))$ denotes the pixel coordinates of LiDAR point $\mathbf{p}$ in the RGB image. The mask centroid is used rather than the centroid of the enclosed LiDAR points because the segmentation mask may include points from nearby objects that are close in image space but have substantially different image depths.

The two detection results are then cross-validated for *false-positive suppression* and *false-negative recovery*. A geometric candidate $\mathbf{p}_{c,k}^{\text{geo}}$ is retained only if a sufficient fraction of its neighboring LiDAR points project within a dynamic-obstacle mask $M_{\text{dyn}}^{(k)}$, thereby rejecting geometrically similar static objects. Conversely, dynamic obstacles indicated by $M_{\text{dyn}}^{(k)}$ but missed by geometric extraction are recovered when they cannot be distinguished from nearby static obstacles using LiDAR geometry alone. For each recovered detection, $\mathbf{p}_{c,k}^{\text{image}}$ is added as its dynamic-obstacle position. The retained geometric detections and recovered image-based detections together form $\mathcal{P}_{\text{dyn}}$.

Finally, $\mathcal{P}_{\text{dyn}}$ is provided to the obstacle velocity estimator of [29] to obtain $\dot{\mathbf{p}}_{c,k}$. The resulting multisensory detection pipeline outputs tracked dynamic obstacles represented by

$$s_k = (\mathbf{p}_{c,k}, \dot{\mathbf{p}}_{c,k}). \quad (11)$$

C. Occupancy Grid Map Construction

The occupancy grid mapping module constructs a 2D representation of the surrounding free and occupied space using ray tracing. Standard ray-tracing updates can leave stale occupied cells after a dynamic obstacle moves away, unnecessarily restricting the feasible control space and potentially resulting in infeasible or suboptimal actions. Tracked dynamic-obstacle states from Eq. (11) in Sec. III-B are therefore incorporated to explicitly remove vacated occupancy.

For standard occupancy updates, the incoming 3D LiDAR point cloud $\mathcal{P}_{\text{lidar}}$ is first filtered along the z-axis to exclude floor and ceiling points and projected onto the horizontal plane as $\mathcal{P}_{\text{lidar}}^{xy}$. The projected points are discretized into N beams spanning 360° . For each beam, the grid cells traversed between the robot position and the endpoint are determined using Bresenham's line algorithm [31]. The occupancy grid is initialized with $\ell_{\text{init}} = 0$, and free and endpoint cells are updated with $\ell_{\text{free}} < 0$ and $\ell_{\text{occ}} > 0$, respectively, following the binary Bayesian occupancy formulation [32]. The log-odds values are bounded within $[\ell_{\text{min}}, \ell_{\text{max}}]$, and cells with $\ell > \ell_{\text{thr}}$ are classified as occupied.

For each tracked dynamic obstacle k , a circular mask centered at its estimated position $\mathbf{p}_{c,k}$ with radius r_k is constructed at each update cycle. Cells within a dynamic-obstacle mask in the previous cycle but outside all such masks in the current cycle are identified as vacated and reset to ℓ_{min} . This targeted reset accelerates the removal of vacated dynamic occupancy while preserving standard occupancy updates for static obstacles.

D. Online Environment Representation Learning

This module integrates the occupancy grid and tracked dynamic-obstacle states to construct continuous obstacle representations for use by the MCBF-QP controller. The processing consists of three stages: (i) obstacle clustering, (ii) dynamic-obstacle separation, and (iii) distance-field learning. Obstacle clusters are first extracted from the occupancy grid maintained as described in Section III-C, then separated according to their association with tracked dynamic obstacles or environment infrastructure, and finally converted into suitable distance-field representations.

1) *Obstacle Clustering*: Occupied grid cells within a radius r_{proc} of the robot are first retained to limit the computational cost of subsequent processing while preserving sufficient environmental context for navigation toward the goal. These cells are clustered using density-based spatial clustering of applications with noise (DBSCAN) [33], with obstacle connectivity determined according to the robot's effective volume. For each cluster, a boundary contour is extracted from its binary occupancy mask using the marching squares algorithm [34]. This boundary representation removes redundant interior cells while preserving the obstacle geometry, which is important for subsequent preprocessing before distance-field learning. The contours are converted from grid indices to xy coordinates and collected as

$$\mathbb{P} = \left\{ \mathcal{P}_j^{(xy)} \right\}, \quad (12)$$

where $\mathcal{P}_j^{(xy)}$ denotes the 2D boundary point set of the j -th cluster.

2) *Dynamic Obstacle Separation*: The tracked dynamic-obstacle states $\{s_k\}$ from Sec. III-B are used to separate dynamic obstacles from environment infrastructure. For each boundary point set $\mathcal{P}_j^{(xy)} \in \mathbb{P}$, points within a radius r_{dyn} of any tracked dynamic obstacle are assigned to $\mathbb{P}_{\text{dyn}}$, while the remaining points are assigned to $\mathbb{P}_{\text{env}}$. Specifically,

$$\mathbb{P}_{\text{dyn}} = \bigcup_j \left\{ \mathbf{p}^{(xy)} \in \mathcal{P}_j^{(xy)} \mid \min_k \|\mathbf{p}^{(xy)} - \mathbf{p}_{c,k}\|_2 \leq r_{\text{dyn}} \right\}, \quad (13)$$

$$\mathbb{P}_{\text{env}} = \bigcup_j \left\{ \mathbf{p}^{(xy)} \in \mathcal{P}_j^{(xy)} \mid \min_k \|\mathbf{p}^{(xy)} - \mathbf{p}_{c,k}\|_2 > r_{\text{dyn}} \right\}. \quad (14)$$

Clusters containing fewer than N_{min} points after this separation are discarded as noise.

3) *Distance Field Learning*: The obstacle point sets, $\mathcal{P}_j^{(xy)}$, are then converted into distance fields for use by the MCBF-QP controller in Section. II-C. To enable real-time computation, GPDF construction and evaluation are implemented in JAX with just-in-time (JIT) compilation. Since the GP covariance matrix dimensions depend on the point-set cardinality, varying point-set sizes prevent reuse of the JIT-compiled routines and incur repeated compilation overhead. Each point set is therefore resampled to a fixed cardinality n_s :

$$\mathcal{P}_j^{(xy)} = \begin{cases} \text{SampleUniform}(\mathcal{P}_j^{(xy)}, n_s), & |\mathcal{P}_j^{(xy)}| > n_s, \\ \mathcal{P}_j^{(xy)} \cup \text{Jitter}(\mathcal{P}_j^{(xy)}, n_s - |\mathcal{P}_j^{(xy)}|), & |\mathcal{P}_j^{(xy)}| < n_s, \\ \mathcal{P}_j^{(xy)}, & |\mathcal{P}_j^{(xy)}| = n_s, \end{cases} \quad (15)$$

after removing coincident points. For oversized point sets, SampleUniform uniformly samples n_s points from $\mathcal{P}_j^{(xy)}$. Recall that $\mathcal{P}_j^{(xy)}$ is filtered to contain only obstacle boundary points during cluster construction. This is important because uniformly sampling from the filled occupied region may retain a substantial number of interior points when the number of occupied cells $|\mathcal{P}_j^{(xy)}|$ is much larger than n_s , resulting in a sampled representation that poorly captures the obstacle geometry. For undersized point sets, Jitter generates the required additional points by sampling $n_s - |\mathcal{P}_j^{(xy)}|$ existing boundary points and perturbing them with additive Gaussian noise [35].

All obstacle point sets in $\mathbb{P}_{\text{env}}$ and those in $\mathbb{P}_{\text{dyn}}$ with sufficient spatial extent are represented by GPDFs, with a separate GPDF constructed for each obstacle [25]. For an obstacle point set $\mathcal{P}_j^{(xy)}$, the covariance between two points $\mathbf{p}^{(xy)}$ and $\mathbf{p}_q^{(xy)}$ is defined by the exponential kernel

$$k_o(\mathbf{p}^{(xy)}, \mathbf{p}_q^{(xy)}) = \exp\left(-\frac{\|\mathbf{p}^{(xy)} - \mathbf{p}_q^{(xy)}\|_2}{L}\right), \quad (16)$$

where L determines the spatial decay of the covariance. At a query position $\mathbf{p}^{(xy)}$, the predicted latent field $\bar{o}(\mathbf{p}^{(xy)})$ is

transformed into a distance field as

$$f_{\text{dist}}(\mathbf{p}^{(xy)}) = -L \log(\bar{\sigma}(\mathbf{p}^{(xy)})) + \text{offset}, \quad (17)$$

where a small offset is introduced to improve numerical stability near obstacle boundaries.

For small-scale dynamic obstacles, such as humans, sparse LiDAR provides insufficient boundary information for accurate GPDF estimation. These obstacles are therefore represented as circles using the estimated center position $\mathbf{p}_{c,k}$ and radius r_k from Section. III-B. The corresponding distance field at a query position $\mathbf{p}^{(xy)}$ is

$$f_{\text{dist}}(\mathbf{p}^{(xy)}) = \|\mathbf{p}^{(xy)} - \mathbf{p}_{c,k}\|_2 - r_k. \quad (18)$$

E. Perception-Based Adaptive MCBF-QP

The proposed MCBF controller adapts its collision-avoidance constraints and geodesic guidance to the local navigation condition. The barrier function and geodesic guidance are first formulated from the perceived environment, after which the local condition determines the collision-avoidance stage and corresponding controller parameters.

1) *Perception-Based MCBF Formulation:* The barrier function for CBF-based collision avoidance is formulated as

$$h(\mathbf{x}) = f_{\text{dist}}(\mathbf{p}) + f_{\text{other}}(\mathbf{x}) + \delta, \quad (19)$$

where $f_{\text{other}}(\mathbf{x})$ accounts for state-dependent terms not captured by the distance field, including non-Euclidean state variables, and δ is the safety margin. With the safe set defined by $\{\mathbf{x} : h(\mathbf{x}) \geq 0\}$, the barrier-function derivative is constrained by $\dot{h}(\mathbf{x}, \mathbf{u}) \geq -\alpha(h(\mathbf{x}))$, as in Eq. (6). The barrier-function derivative associated with environment infrastructure is

$$\dot{h}_{\text{env}} = \nabla_{\mathbf{p}} f_{\text{dist}}^{\top} (f_p(\mathbf{x}) + g_p(\mathbf{x})\mathbf{u}) + \nabla_{\mathbf{x}} f_{\text{other}}^{\top} (f(\mathbf{x}) + g(\mathbf{x})\mathbf{u}),$$

while the motion of dynamic obstacle k , with estimated velocity $\dot{\mathbf{p}}_{c,k}$, introduces the relative-velocity term

$$\begin{aligned} \dot{h}_{\text{dyn}} = & \nabla_{\mathbf{p}} f_{\text{dist}}^{(k)\top} (f_p(\mathbf{x}) + g_p(\mathbf{x})\mathbf{u} - \dot{\mathbf{p}}_{c,k}) \\ & + \nabla_{\mathbf{x}} f_{\text{other}}^{\top} (f(\mathbf{x}) + g(\mathbf{x})\mathbf{u}). \end{aligned} \quad (20)$$

Here, $f_p(\mathbf{x})$ and $g_p(\mathbf{x})$ denote the components of $f(\mathbf{x})$ and $g(\mathbf{x})$ corresponding to the position dynamics.

In addition to the CBF constraint, the MCBF controller employs a geodesic tangential guidance vector ϕ to encourage obstacle-circumventing motion. To avoid transient changes in guidance caused by dynamic obstacles, ϕ is computed using only environment-infrastructure obstacles. Specifically, h^{ϕ} is selected as the barrier function associated with the closest environment-infrastructure cluster obstructing the direct path between the robot and the target. Following [24], let $e^{(0)}$ be one of m uniformly sampled candidate directions satisfying $e^{(0)} \in \mathcal{N}(\nabla_{\mathbf{x}} h^{\phi}(\mathbf{x}))$. Each candidate generates a path $[\mathbf{x}^{(0)}, \mathbf{x}^{(1)}, \dots, \mathbf{x}^{(N)}]$ along the isoline $h^{\phi}(\mathbf{x}) = h^{\phi}(\mathbf{x}^{(0)})$ using a first-order boundary approximation. The candidate with the minimum accumulated potential $C^{(N)}$

determines $\phi(\mathbf{x})$ through its initial direction $e^{(0)}$. The state, direction, and accumulated potential are updated as

$$\begin{aligned} \mathbf{x}^{(i+1)} &= \mathbf{x}^{(i)} + \beta H(\mathbf{x}^{(i)}) H(\mathbf{x}^{(i)})^{\top} e^{(i)}, \\ e^{(i+1)} &= \frac{H(\mathbf{x}^{(i)}) H(\mathbf{x}^{(i)})^{\top} e^{(i)}}{\|H(\mathbf{x}^{(i)}) H(\mathbf{x}^{(i)})^{\top} e^{(i)}\|_2}, \\ C^{(i+1)} &= C^{(i)} + \beta c(\mathbf{x}^{(i+1)}, \mathbf{p}_g^{(xy)}). \end{aligned} \quad (21)$$

Here, β is the step size approximated from the obstacle-cluster boundary length, $H(\mathbf{x})$ contains an orthonormal basis of the tangent hyperplane to the corresponding isoline, and $c(\mathbf{x}, \mathbf{p}_g^{(xy)})$ is a user-defined penalty function that evaluates the desirability of a state relative to the target position.

2) *Stage Selection:* The MCBF controller requires stage-dependent adaptation of both the CBF constraint and geodesic guidance. Conservative CBF settings improve dynamic-obstacle clearance but can impede narrow-passage traversal, while boundary-based geodesic guidance is necessary to capture narrow passages but can poorly represent the robot state in open space. We therefore define three collision-avoidance stages—dynamic-obstacle, narrow-space, and normal—and adapt the CBF parameters and geodesic guidance accordingly.

The dynamic-obstacle stage is selected if any dynamic obstacle k satisfies

$$f_{\text{dist}}^{(k)}(\mathbf{p}_{\text{rob}}^{(xy)}) < r_{\text{close}}, \quad \nabla_{\mathbf{p}_{c,k}} f_{\text{dist}}^{(k)}(\mathbf{p}_{\text{rob}}^{(xy)})^{\top} \dot{\mathbf{p}}_{c,k} < -v_{\text{close}},$$

where $v_{\text{close}} > 0$ is a predefined closing-speed threshold. These conditions identify a dynamic obstacle that is both within the specified proximity and approaching the robot at a rate requiring immediate avoidance.

If the dynamic-obstacle criterion is not satisfied, the narrow-space stage is evaluated using two complementary indicators derived from $\mathbb{P}_{\text{env}}$. The inter-cluster indicator identifies narrow passages based on the minimum separation between distinct environment-infrastructure clusters. This criterion is inapplicable when the relevant boundaries are represented by a single cluster, as in enclosed regions with a narrow opening through which the robot must enter or exit. These configurations are termed *confinement* and are identified using a separate confinement indicator. Dynamic obstacles in $\mathbb{P}_{\text{dyn}}$ are excluded from both indicators, as their transient geometry is not used to characterize the underlying environment structure.

For the inter-cluster indicator, a rectangular region $\mathcal{R}$ of length L and width W is defined to capture the environment near the robot and along its current navigation direction. The region is centered at $\mathbf{p}_{\text{rob}}^{(xy)}$ and oriented along

$$\mathbf{s} = \begin{cases} \phi(\mathbf{x}), & \text{if } \phi(\mathbf{x}) \text{ is defined,} \\ [\cos \theta_{\text{rob}}, \sin \theta_{\text{rob}}]^{\top}, & \text{otherwise,} \end{cases} \quad (22)$$

where θ_{rob} is the robot orientation. The minimum separation between distinct environment-infrastructure clusters within $\mathcal{R}$ is defined as

$$r_{\text{pass}} = \min_{\substack{\mathcal{P}_i^{(xy)}, \mathcal{P}_j^{(xy)} \in \mathbb{P}_{\text{env}} \\ i \neq j}} \left\{ \|\mathbf{p}_i - \mathbf{p}_j\|_2 \mid \mathbf{p}_i \in \mathcal{P}_i^{(xy)} \cap \mathcal{R}, \mathbf{p}_j \in \mathcal{P}_j^{(xy)} \cap \mathcal{R} \right\}.$$

The narrow-space stage is activated when $r_{\text{pass}} < r_{\text{narrow}}$.

The confinement indicator evaluates whether a point $\mathbf{p}_{\text{confine}}^{(xy)}$ is enclosed by a single environment-infrastructure cluster with a narrow opening. Each point $\mathbf{p}^{(xy)} \in \mathbb{P}_{\text{env}}$ is assigned to one of N_{bin} uniformly spaced bearing bins according to

$$\text{bin}(\mathbf{p}^{(xy)}) = \left\lfloor \frac{\text{atan2}(\mathbf{p}^{(y)} - \mathbf{p}_{\text{confine}}^{(y)}, \mathbf{p}^{(x)} - \mathbf{p}_{\text{confine}}^{(x)}) + \pi}{2\pi/N_{\text{bin}}} \right\rfloor.$$

Within each bearing bin, the environment-infrastructure point nearest to $\mathbf{p}_{\text{confine}}^{(xy)}$ is retained, and its distance is denoted by d_{bin} . A bin is blocked if a nearest point exists and $d_{\text{bin}} < d_{\text{confined}}$. The point $\mathbf{p}_{\text{confine}}^{(xy)}$ is classified as confined when at least 90% of the N_{bin} bins are blocked. When the robot is confined, adjacent bearing bins are examined for a discontinuity in d_{bin} ; a difference exceeding a predefined threshold identifies an opening in the cluster enclosing the robot.

The confinement test is first evaluated at the robot position, $\mathbf{p}_{\text{confine}}^{(xy)} = \mathbf{p}_{\text{rob}}^{(xy)}$. If the robot is confined, the narrow-space stage is activated and a temporary target is placed at the detected opening farthest from the robot to facilitate departure from the enclosure. Once the robot is no longer classified as confined, the temporary target is discarded and the original target $\mathbf{p}_g^{(xy)}$ is restored. The confinement test is then evaluated at the target position, $\mathbf{p}_{\text{confine}}^{(xy)} = \mathbf{p}_g^{(xy)}$. If the robot is not confined but the target is, the narrow-space stage is activated when the robot is sufficiently close to $\mathbf{p}_g^{(xy)}$ to facilitate entry into the enclosure. Otherwise, the normal stage is selected.

3) *Stage-Dependent MCBF Adaptation*: The selected stage adapts the MCBF controller in two aspects: CBF constraint conservativeness and geodesic tangential guidance. First, the CBF conservativeness is adjusted through the class- $\mathcal{K}$ function $\alpha(h)$ and the safety margin δ . The dynamic-obstacle, normal, and narrow-space stages use progressively less conservative settings:

$$\alpha_{\text{dyn}}(h) \leq \alpha_{\text{normal}}(h) \leq \alpha_{\text{narrow}}(h), \quad \delta_{\text{narrow}} \leq \delta_{\text{normal}} \leq \delta_{\text{dyn}}.$$

Second, the selected stage determines the isoline on which the geodesic tangential guidance vector ϕ is evaluated. In the dynamic-obstacle and normal stages, the geodesic approximation is initialized at the robot state,

$$\mathbf{x}^{(0)} = \mathbf{x}_{\text{rob}}, \quad (23)$$

and therefore follows the isoline corresponding to the robot's current position. In the narrow-space stage, this isoline may exclude passages narrower than approximately $2h^\phi(\mathbf{x}_{\text{rob}})$. The robot state is therefore projected onto the zero isoline, and the resulting projection $\mathbf{x}_{\text{rob}}^{\text{proj}}$ is used as the initial state:

$$\mathbf{x}^{(0)} = \mathbf{x}_{\text{rob}}^{\text{proj}}, \quad h^\phi(\mathbf{x}^{(0)}) = 0. \quad (24)$$

These stage-dependent adaptations reduce CBF conservativeness in confined regions while preserving obstacle-circumventing geodesic guidance and increased clearance from rapidly approaching dynamic obstacles.

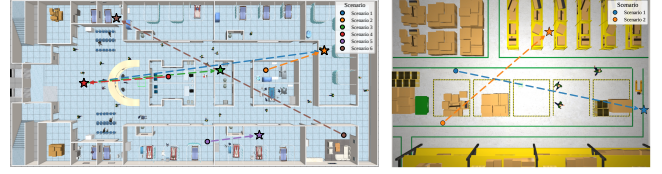


Fig. 2: Hospital (left) and warehouse (right) simulation.

IV. EXPERIMENTS

The proposed memory-aware multi-sensor perception system with MCBF-QP is evaluated in simulation and on hardware. The simulation study, presented in Section IV-A, compares the proposed method with an ablated variant and three baseline methods to quantify its safety and liveness performance, while the hardware experiment, presented in Section IV-B.1, demonstrates deployment feasibility on a differential-drive Scout robot.

A. Simulation Experiments

1) *Experiment Design*: Two simulated environments are adapted from the AWS RoboMaker Warehouse World and Hospital World [36]. The robot is a Freight differential-drive base equipped with a 360° 3D LiDAR and four RGB cameras, each with a 90° field of view, providing full 360° visual coverage. The camera-based dynamic obstacle detector is trained specifically for human detection; other movable obstacle classes can be incorporated by extending the training dataset with the corresponding categories. Six navigation scenarios are defined in the hospital environment and two in the smaller, less cluttered warehouse environment, with 20 and 3 human actors as dynamic obstacles per trial, respectively. Each scenario specifies fixed start and goal poses, with all scenarios shown in Fig. 2. Each human follows an independently designed waypoint sequence without reacting to the robot and is initialized at a random position along its trajectory at the beginning of each trial, resulting in varying robot-human encounter timings. Each scenario is repeated five times for each baseline method to obtain performance measures that are less sensitive to the particular random initialization.

2) *Ablation and Baselines*: The proposed adaptive MCBF-QP is compared with an ablated variant, denoted *Adaptive CBF*, which retains the same perception, obstacle representation, and parameter adaptation pipeline but replaces the MCBF-QP formulation with a standard CBF-QP. Three existing perception-based navigation approaches are also considered: *MPC-CBF* [16], which cluster LiDAR point clouds into fitted ellipses; *DWA* [37], operating on a standard occupancy map; and *CE-CBF* [18], a CBF that learns local obstacle representations from LiDAR data using a GP and incorporates a circulation constraint to enhance liveness. *MPC-CBF* and *CE-CBF* do not retain previously observed environmental information, whereas *DWA* uses a memory-aware occupancy map constructed through ray tracing.

3) *Metrics*: All methods are evaluated under identical scenarios and sensing conditions using success rate, human

TABLE I: Comparison of the memory-aware perception-based MCBF, its ablated CBF variant, and baselines in target-reaching success rate, collision rate per human encountered, and trajectory tortuosity. [†]MPC-CBF was infeasible at the initial position; tortuosity is undefined.

Env.	Sc.	Adaptive MCBF (Ours)			Adaptive CBF			CE-CBF			MPC-CBF			DWA		
		Succ. %	Coll./Enc. (%)	Tort.	Succ. %	Coll./Enc. (%)	Tort.	Succ. %	Coll./Enc. (%)	Tort.	Succ. %	Coll./Enc. (%)	Tort.	Succ. %	Coll./Enc. (%)	Tort.
Hospital	1	100.0	4/56 (7.1)	1.63	0.0	5/60 (8.3)	4.63	0.0	9/55 (14.8)	7.42	0.0	5/29 (17.2)	1.10	100.0	4/56 (7.1)	1.53
	2	100.0	3/32 (9.4)	3.07	0.0	0/0	129.53	0.0	0/0	51.48	0.0	0/0	1.10	0.0	0/0	1.08
	3	100.0	1/20 (5.0)	1.37	40.0	1/39 (2.6)	2.94	0.0	4/60 (6.7)	23.90	0.0	3/24 (12.5)	1.13	60.0	1/17 (5.9)	1.32
	4	100.0	2/27 (7.4)	1.95	40.0	2/33 (6.1)	5.33	0.0	0/0	99.26	0.0	0/0	1.12	0.0	0/3 (0.0)	2.59
	5	100.0	0/14 (0.0)	3.61	80.0	6/50 (12.0)	8.97	0.0	2/15 (13.3)	52.40	0.0	0/0	— [†]	0.0	2/8 (25.0)	1.70
	6	100.0	8/45 (17.8)	1.70	20.0	9/79 (11.4)	4.71	0.0	1/67 (1.5)	16.38	0.0	1/1 (100.0)	1.05	80.0	2/50 (4.0)	3.34
Warehouse	1	100.0	0/11 (0.0)	1.75	80.0	0/10 (0.0)	1.39	100.0	1/13 (7.7)	1.73	0.0	1/9 (11.1)	1.19	100.0	0/6 (0.0)	1.07
	2	100.0	0/10 (0.0)	2.28	40.0	1/12 (8.3)	4.89	0.0	3/44 (6.8)	23.60	0.0	0/0	— [†]	100.0	0/2 (0.0)	1.19
Summary		100.0	18/215 (8.4)	2.17	37.5	24/283 (8.5)	20.30	12.5	20/254 (7.9)	34.52	0.0	10/63 (15.9)	1.12	55.0	9/142 (6.4)	1.73

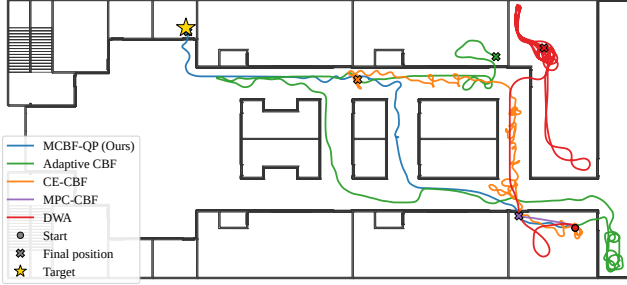


Fig. 3: Representative navigation trajectories of the evaluated controllers in Scenario 6 of the hospital environment.

collision rate, and trajectory tortuosity. Success rate is measured as the percentage of trials in which the robot reaches the goal, while human collision rate is the proportion of human encounters resulting in a collision, with encounters determined by the ground-truth robot–human distance being within three meters. Trajectory tortuosity is defined as the traveled path length divided by the straight-line distance between the initial and final positions, with larger values indicating greater oscillation and detouring within a scenario.

4) *Result Analysis*: Table I shows that the proposed MCBF-QP is the only method to achieve a 100% success rate while maintaining a comparable human collision rate. The memory-unaware *CE-CBF* and *MPC-CBF*, which construct obstacle representations solely from current observations without retaining previously observed environmental information, achieve the lowest success rates ($< 12.5\%$). *MPC-CBF* approximates LiDAR observations with fitted ellipses for real-time control, but the conservative approximation frequently renders the controller infeasible. *CE-CBF* introduces a circulation constraint for liveness, but its locally computed guidance is sensitive to LiDAR point-cloud distributions and can provide inconsistent directions that hinder target convergence. These results indicate that liveness enhancement alone is insufficient without persistent environmental information.

The memory-aware *Adaptive CBF* and *DWA* achieve higher success rates (30% – 50%) by retaining previously observed environmental information, but neither incorporates a liveness-enhancement strategy. Consequently, they remain susceptible to local undesirable equilibria and limit cycles, indicating that persistent environmental information alone is also insufficient for reliable target convergence.

Across the baselines, dynamic obstacles can further push

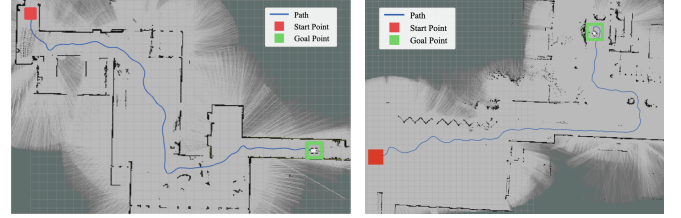


Fig. 4: Robot trajectories and online occupancy maps for indoor (left) and outdoor (right) hardware experiments.

the robot away from the target, resulting in long detours and high trajectory tortuosity even when the target is reached, as illustrated in Fig. 3. The proposed method combines persistent infrastructure mapping of explored regions with a modified MCBF-QP that uses the perceived environment to adapt the guiding direction and parameters, enabling efficient convergence toward the target.

B. Hardware Experiment

1) *Platform and Computing Architecture*: The hardware platform consists of a Scout 2.0 skid-steer mobile base equipped with a rear-mounted SICK multiScan136 3D LiDAR, a front-mounted Intel RealSense D435 RGB-D camera to provide additional depth observations in the near-field where LiDAR coverage is sparse, and a rear-mounted Logitech C920 RGB camera for human detection. All sensors are rigidly mounted with fixed extrinsic calibration. The complete system pipeline runs onboard an NVIDIA Jetson AGX Orin.

2) *Navigation Experiment*: We qualitatively evaluated the complete system in two previously unmapped real-world environments. The indoor lobby and outdoor courtyard both contain narrow passages, concave structures, moving pedestrians, sloped terrain, and irregular structures or vegetation. In both environments, the robot constructed the occupancy map and distance fields online and successfully reached the goal. Fig. 4 overlays the onboard-estimated trajectory on the resulting occupancy map, demonstrating the end-to-end feasibility of the proposed system using only onboard sensing and computation.

V. DISCUSSIONS AND FUTURE WORK

This work presented a memory-aware, multi-sensor navigation framework that combines an online persistent environmental representation with stage-adaptive MCBF-QP control

to enable safety and liveness for navigation in previously un-mapped, dynamic indoor/outdoor environments. Future work could investigate the increasing computational burden of processing larger occupancy maps, which can slow controller execution under limited onboard computing resources, potentially through adaptive memory management and persistent representation. Another direction is to further improve dynamic obstacle avoidance by incorporating accurate obstacle motion prediction, with the goal of achieving collision-free navigation in complex and crowded environments.

REFERENCES

- [1] S. LaValle, "Rapidly-exploring random trees: A new tool for path planning," *Research Report 9811*, 1998.
- [2] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [3] D. Dolgov, S. Thrun, M. Montemerlo, and J. Diebel, "Practical search techniques in path planning for autonomous driving," *ann arbor*, vol. 1001, no. 48105, pp. 18–80, 2008.
- [4] B. Lütjens, M. Everett, and J. P. How, "Safe reinforcement learning with model uncertainty estimates," in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 8662–8668.
- [5] K. Zhu, B. Li, W. Zhe, and T. Zhang, "Collision avoidance among dense heterogeneous agents using deep reinforcement learning," *IEEE Robotics and Automation Letters*, vol. 8, no. 1, pp. 57–64, 2022.
- [6] M. Jiang, Y. Li, S. Zhang, S. Chen, C. Wang, and M. Yang, "Hope: A reinforcement learning-based hybrid policy path planner for diverse parking scenarios," *IEEE Transactions on Intelligent Transportation Systems*, vol. 26, no. 5, pp. 6130–6141, 2025.
- [7] G. Williams, A. Aldrich, and E. A. Theodorou, "Model predictive path integral control: From theory to parallel computation," *Journal of Guidance, Control, and Dynamics*, vol. 40, no. 2, pp. 344–357, 2017.
- [8] S. M. Khansari-Zadeh and A. Billard, "A dynamical system approach to realtime obstacle avoidance," *Autonomous Robots*, vol. 32, pp. 433–454, 2012.
- [9] L. Huber, A. Billard, and J.-J. Slotine, "Avoidance of convex and concave obstacles with convergence ensured through contraction," *IEEE Robotics and Automation Letters*, vol. 4, no. 2, pp. 1462–1469, 2019.
- [10] C. K. Fourie, N. Figueroa, and J. A. Shah, "On-manifold strategies for reactive dynamical system modulation with nonconvex obstacles," *IEEE Transactions on Robotics*, vol. 40, pp. 2390–2409, 2024.
- [11] J. Zeng, B. Zhang, and K. Sreenath, "Safety-critical model predictive control with discrete-time control barrier function," in *2021 American Control Conference (ACC)*. IEEE, 2021, pp. 3882–3889.
- [12] A. Srikanthan, Y. Xue, V. Kumar, N. Matni, and N. Figueroa, "Admm-mcbf-lca: A layered control architecture for safe real-time navigation," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 9243–9250.
- [13] Y. Xue, Z. Zhang, K. Åkesson, and N. Figueroa, "Proactive local-minima-free robot navigation: Blending motion prediction with safe control," *IEEE Robotics and Automation Letters*, vol. 11, no. 4, pp. 5096–5103, 2026.
- [14] A. Pfrunder, P. V. K. Borges, A. R. Romero, G. Catt, and A. Elfes, "Real-time autonomous ground vehicle navigation in heterogeneous environments using a 3d lidar," in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 2601–2608.
- [15] Z. Liu, A. Amini, S. Zhu, S. Karaman, S. Han, and D. L. Rus, "Efficient and robust lidar-based end-to-end navigation," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 13 247–13 254.
- [16] P. Tooranjipour and B. Kiumarsi, "Lidar-based model predictive control using control barrier functions," in *2025 American Control Conference (ACC)*, 2025, pp. 315–322.
- [17] H. Dong, C.-Y. Weng, C. Guo, H. Yu, and I.-M. Chen, "Real-time avoidance strategy of dynamic obstacles via half model-free detection and tracking with 2d lidar for mobile robots," *IEEE/ASME Transactions on Mechatronics*, vol. 26, no. 4, pp. 2215–2225, 2021.
- [18] V. M. Gonçalves, P. Krishnamurthy, A. Tzes, and F. Khorrami, "Control barrier functions with circulation inequalities," *IEEE Transactions on Control Systems Technology*, vol. 32, no. 4, pp. 1426–1441, 2024.
- [19] M. Tong, C. Dawson, and C. Fan, "Enforcing safety for vision-based controllers via control barrier functions and neural radiance fields," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 10 511–10 517.
- [20] S. Das, Y. Xue, H. Li, and N. Figueroa, "Rnbf: Real-time rgb-d based neural barrier functions for safe robotic navigation," in *2026 IEEE/SICE International Symposium on System Integration (SII)*, 2026, pp. 537–544.
- [21] Z. Xu, X. Zhan, Y. Xiu, C. Suzuki, and K. Shimada, "Onboard dynamic-object detection and tracking for autonomous robot navigation with rgb-d camera," *IEEE Robotics and Automation Letters*, vol. 9, no. 1, pp. 651–658, 2024.
- [22] R. Pieroni, S. Specchia, M. Corno, and S. M. Savaresi, "Multi-object tracking with camera-lidar fusion for autonomous driving," in *2024 European Control Conference (ECC)*, 2024, pp. 2774–2779.
- [23] Z. Xu, H. Shen, X. Han, H. Jin, K. Ye, and K. Shimada, "Lv-dot: Lidar-visual dynamic obstacle detection and tracking for autonomous robot navigation," *arXiv preprint arXiv:2502.20607*, 2025.
- [24] Y. Xue and N. Figueroa, "No minima, no collisions: Combining modulation and control barrier function strategies for feasible dynamical collision avoidance," 2025. [Online]. Available: <https://arxiv.org/abs/2502.14238>
- [25] H. J. Choi and N. Figueroa, "Towards feasible dynamic grasping: Leveraging gaussian process distance field, se(3) equivariance, and riemannian mixture models," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 6455–6461.
- [26] C. Le Gentil, O.-L. Ouabi, L. Wu, C. Pradalier, and T. Vidal-Calleja, "Accurate gaussian-process-based distance fields with applications to echolocation and mapping," *IEEE Robotics and Automation Letters*, vol. 9, no. 2, pp. 1365–1372, 2023.
- [27] W. Xu, Y. Cai, D. He, J. Lin, and F. Zhang, "FAST-LIO2: Fast direct LiDAR-Inertial odometry," *IEEE Transactions on Robotics*, vol. 38, no. 4, pp. 2053–2073, 2022.
- [28] T. Moore and D. Stouch, "A generalized extended kalman filter implementation for the robot operating system," in *Intelligent Autonomous Systems 13*, ser. Advances in Intelligent Systems and Computing, E. Menegatti, N. Michael, K. Berns, and H. Yamaguchi, Eds. Cham, Switzerland: Springer, 2016, vol. 302, pp. 335–348.
- [29] M. Przybyła, "Detection and tracking of 2d geometric obstacles from lrf data," in *Proceedings of the 11th International Workshop on Robot Motion and Control (RoMoCo)*. Wasowo Palace, Poland: IEEE, July 3–5 2017, pp. 135–141.
- [30] Ultralytics, "Ultralytics yolo26: Unified real-time end-to-end vision models," 2026. [Online]. Available: <https://arxiv.org/abs/2606.03748>
- [31] J. E. Bresenham, "Algorithm for computer control of a digital plotter," *IBM Systems Journal*, vol. 4, no. 1, pp. 25–30, 1965.
- [32] A. Elfes, "Using occupancy grids for mobile robot perception and navigation," *Computer*, vol. 22, no. 6, pp. 46–57, 1989.
- [33] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, 1996, pp. 226–231.
- [34] C. Maple, "Geometric design and space planning using the marching squares and marching cube algorithms," in *Proceedings of the 2003 International Conference on Geometric Modeling and Graphics*, 2003, pp. 90–95.
- [35] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, "Pointnet++: Deep hierarchical feature learning on point sets in a metric space," *Advances in neural information processing systems*, vol. 30, 2017.
- [36] Amazon Web Services, "Aws robomaker hospital world and small warehouse world," <https://github.com/aws-robotics/aws-robomaker-hospital-world>; <https://github.com/aws-robotics/aws-robomaker-small-warehouse-world>, accessed: 2026-04-11.
- [37] D. Fox, W. Burgard, and S. Thrun, "The dynamic window approach to collision avoidance," *IEEE Robotics & Automation Magazine*, vol. 4, no. 1, pp. 23–33, Mar. 1997.